Stacks
(Assignment Solutions)

Question 1 :

```
bool isPalin(string str) {
        int n = str.size();
        for(int i=0; i<n/2; i++) {
            if(str[i] != str[n-i-1]) {
                return false;
            }
        }

        return true;
    }

    bool isPalindrome(ListNode* head) {
        string str = "";
        ListNode* temp = head;

        while(temp != NULL) {
            str += temp->val;
            temp = temp->next;
        }

        return isPalin(str);
    }
```

Question 2 :

```
string decodeString(string s) {
        stack<int> st;
        stack<string> st1;
        string sb;
        int n = 0;

        for (char c : s) {
            if (isdigit(c)) {
```

```
                n = n * 10 + (c - '0');
            } else if (c == '[') {
                st.push(n);
                n = 0;
                st1.push(sb);
                sb = "";
            } else if (c == ']') {
                int k = st.top();
                st.pop();
                string temp = sb;
                sb = st1.top();
                st1.pop();
                while (k-- > 0) {
                    sb += temp;
                }
            } else {
                sb += c;
            }
        }

        return sb;
    }
```

Question 3 :

```
string simplifyPath(string path) {
        string ok;
        vector<string>store;
        int i = 1, n = s.size();
        while(i < n)
        {
            while(s[i]=='/') i++;
            while(s[i] and s[i] != '/') ok += s[i++];

            if(ok == ".." and store.size()) store.pop_back();
                        else if(ok.size() and ok != "." and ok != "..")
store.push_back(ok);
            ok.clear();
        }
```

```
        ok = "/";
        for(auto str:store) ok += str, ok += "/";
        if(ok.size() > 1) ok.pop_back();

        return ok;
    }
```

Question 4 :

```
int trap(vector<int>& height) {
        int left = 0;
        int right = height.size() - 1;
        int leftMax = height[left];
        int rightMax = height[right];
        int water = 0;

        while (left < right) {
            if (leftMax < rightMax) {
                left++;
                leftMax = max(leftMax, height[left]);
                water += leftMax - height[left];
            } else {
                right--;
                rightMax = max(rightMax, height[right]);
                water += rightMax - height[right];
            }
        }

        return water;
    }
```